**Università degli Studi di Messina**
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

# HANDS-ON 19:
Federated Learning con Ray.io e PyTorch

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

# Indice

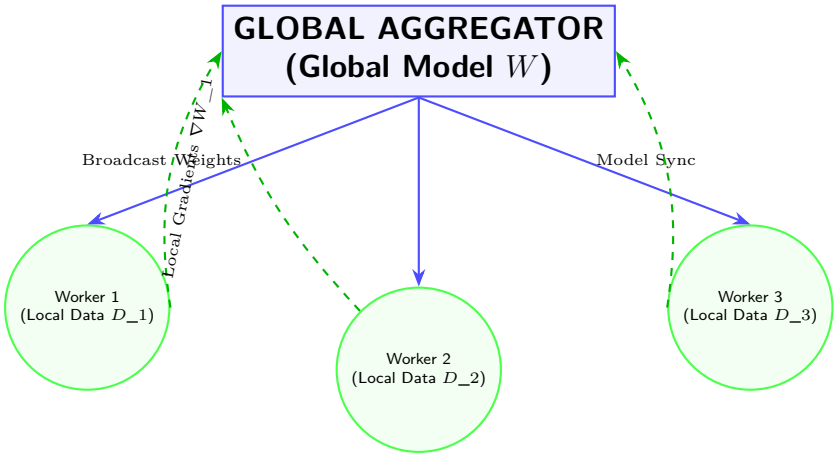
# 1 Problem Statement

L'obiettivo di questo Hands-On è l'implementazione di un sistema di **Federated Learning (FL)** per la classificazione di immagini del dataset MNIST. A differenza del paradigma tradizionale centralizzato, il Federated Learning permette di addestrare modelli di Machine Learning su dati distribuiti, mantenendo la privacy dei dati locali.

In questo esperimento, abbiamo utilizzato **Ray.io** come framework di orchestrazione distribuita e **PyTorch** per la definizione e l'addestramento del modello neurale.

# 2 Stato dell'Arte

Il Federated Learning si basa sul concetto di portare il calcolo dai dati, invece dei dati al calcolo.




*Decentralized Training (Privacy Preserved)*

Figure 1: Ciclo di addestramento nel Federated Learning: Distribuzione pesi e Aggregazione gradienti.

Il processo tipico prevede:

1. Un'entità centrale (**Aggregator**) che invia i pesi del modelo globale ai partecipanti.
2. I partecipanti (**Workers**) che addestrano localmente il modello sui propri dati.
3. L'invio degli aggiornamenti (gradienti o pesi) all'Aggregator.
4. L'aggregazione degli aggiornamenti tramite algoritmi come il **Federated Averaging (FedAvg)**.

L'algoritmo FedAvg calcola la media pesata dei parametri del modello:

$$w_t + 1 = \sum_k = 1^K \frac{n_k}{n} w_t + 1^k \tag{1}$$

dove $n_k$ è il numero di esempi del worker $k$ e $n$ il totale degli esempi.

# 3 Metodologia ed Implementazione

Il sistema è stato implementato con 1 Aggregator e 6 Worker distribuiti come **Ray Actors**.

## 3.1 Partizionamento dei Dati

Le 60.000 immagini del set di training di MNIST sono state suddivise in modo casuale (**IID split**) tra i 6 worker, assegnando 10.000 immagini a ciascuno. Ogni worker gestisce il proprio `DataLoader` locale.

## 3.2 Architettura Distribuita

- **FederatedWorker**: Riceve i pesi globali, esegue 1 epoca di training locale e restituisce i pesi aggiornati.
- **FederatedAggregator**: Coordina i round di comunicazione, esegue l'aggregazione FedAvg e valuta il modello globale su un set di test centrale (10.000 immagini).

## 3.3 Istruzioni di Esecuzione

L'addestramento federato viene gestito tramite Makefile:

**Run**

```
$ make install # Setup ambiente
$ make run     # Avvio training federato
```

# 4 Risultati

Il sistema è stato testato per 3 round di comunicazione. I log mostrano un rapido incremento dell'accuratezza globale:

**Log Addestramento Federato**

```
>>> Initializing Federated Learning with 6 workers...
>>> Round 1 starting...
Round 1 - Global Test Accuracy: 92.48%
>>> Round 2 starting...
Round 2 - Global Test Accuracy: 95.21%
>>> Round 3 starting...
Round 3 - Global Test Accuracy: 96.16%
Training finished.
```

L'accuratezza finale del **96.16%** ottenuta in soli 3 round dimostra l'efficacia dell'approccio federato nella convergenza del modello MLP.

# 5 Conclusioni e Sviluppi Futuri

L'implementazione ha dimostrato come Ray.io semplifichi drasticamente la gestione di attori distribuiti per compiti di intelligenza artificiale. Il Federated Learning si è rivelato un metodo robusto per l'addestramento distribuito, riuscendo a mantenere prestazioni paragonabili alla versione centralizzata pur mantenendo i dati frammentati su worker indipendenti.